Part 1: Reading

Question 1.

GRPO removes the need for a learned value function (critic). Explain how GRPO estimates the baseline for advantage computation without a critic, and what trade-offs this introduces.

After SFT, the resulting policy π_θ can follow instructions and produce reasonable outputs, but has no mechanism to improve based on *outcome quality*. RL addresses this: the model samples outputs, a reward model scores them, and π_θ is updated to maximise expected reward. To update the policy, we must estimate the *advantage* for each output: how much higher or lower was its reward compared to what the model would typically achieve on this prompt? PPO estimates this expected baseline reward via a learned critic; GRPO via group-relative normalisation.

PPO (Schulman et al., 2017) trains a separate 'critic' network V_ψ to serve as the baseline, predicting expected future reward *at every token.* The *advantage* at token t, A_t, is defined as the difference between the actual reward received, r, and the critic's prediction, that is:

$$A_t = r - V_\psi(q,\, o_{<t}),$$

where q and $o_{<t}$ denote the query and all tokens generated prior to token t, respectively. Tokens with $A_t > 0$ lead to better-than-expected outcomes and are reinforced by increasing their probability under π_θ, whilst tokens with $A_t < 0$ are suppressed. This enables fine-grained, per-token credit assignment via Generalised Advantage Estimation (GAE; Schulman et al., 2015).[1] In the LLM setting, however, there are two problems of note:

- The reward model provides only a single scalar r at the end of the sequence. Consequently, V_ψ must learn to predict the final reward from partial sequences during generation, a target for which it has no direct supervision.
- V_ψ is typically of comparable size to π_θ, requiring both models to be in memory simulaneously thoroughout training (Shao et al., 2024).

GRPO (Shao et al., 2024; Skiredj, 2025) eliminates V_ψ. For a query q, the policy samples G complete outputs $\{o_i\}_{i\in[G]}$, passing each one to the reward model, which scores it independently, yielding $\{r_i\}_{i\in[G]}$. The baseline reward is then simply $\bar{r}$, the mean of these scores. Define the *advantage* for each output o_i as:

$$\hat{A}_i = \frac{r_i - \bar{r}}{\sigma_r + \epsilon},$$

where σ_r is the standard deviation of rewards within the group. If $r_i > \bar{r}$, output o_i was better than typical, so reinforce it; if $r_i < \bar{r}$, then suppress it. Every token within o_i inherits the same $\bar{A}_i$.

The trade-offs are:

- Every token in o_i shares the same $\hat{A}_i$, so the model cannot tell which tokens produced success or failure. As noted in Shao et al., GRPO+PS partially fixes this.
- If all G outputs score identically, all advantages vanish, so no learning occurs.
- Large G is expensive, possibly offsetting the memory savings from dropping the critic.

Question 2.

Discuss the difference between using a learned reward model vs. a rule-based reward function (e.g., checking if an answer matches ground truth). What are the risks of reward hacking in each case?

A **learned reward model** r_φ is a neural network trained on human-labelled preference pairs (o^+, o^-) for the same query, where humans indicate that o^+ is preferred; the model learns to assign $r_\varphi(o^+) > r_\varphi(o^-)$. Its main advantage is that it captures nuanced qualities difficult to quantify formally, like coherence, helpfulness, reasoning quality; its main drawback is that it is expensive to

[1] This underpins the standard RLHF pipeline used to train InstructGPT (Ouyang et al., 2022).

train and is imperfect, being an approximation of human preference rather than 'ground-truth.' By exploiting 'blind spots' in r_φ, *i.e.*, outputs that fall outside the distribution of human-labelled pairs, π_θ can produce responses that score highly according to the learned network but that humans would not endorse. One example might be generating confident-sounding but incorrect reasoning that r_φ was never trained to penalise.

By contrast, a **rule-based reward function** is a *deterministic* scoring rule applied directly to the output. In `DeepSeekMath` (Shao et al., 2024), for instance: $r(o) = \mathbf{1}[\text{extracted truth} = \text{ground truth}]$. Certainly, this is cheap, transparent, and directly to the ground truth, but limited to settings in which correctness is ambiguously verifiable. Reward hacking might occur when π_θ can 'game' the specific rule in lieu of learning the intended before. For instance, *if* only the final answer is checked, then π_θ might learn to output a plausible number with no valid reasoning, or blindly pattern-match to answers commonly seen during training.

Question 3.

In Homework 3, you fine-tuned a VLM using supervised fine-tuning (SFT) with LoRA. Compare the SFT approach with GRPO training: how does the training signal differ, and when would each be preferred?

SFT and GRPO are complementary rather than competing approaches, with the former generally applied *using* the latter as initialisation (*e.g.*, In Shao et al., 2024, SFT produces `DeepSeekMath-Instruct`, which GRPO refines into `DeepSeekMath-RL`.) The two differ much in their training signal.

Indeed, SFT is a *supervised* algorithm given a dataset of (input, output) pairs, the model is trained to *reproduce* the reference output via cross-entropy loss, with *no* feedback provided on whether its outputs are actually correct. Whilst this teaches the model the correct format and basic desired behaviour, it penalises any deviation from the reference path, even if the model should produce a valid alternative solution.

GRPO, by contrast, is an *RL* algorithm whose training signal is outcome quality: π_θ generates its own outputs, a reward model scores them, and the policy is updated to maximise the expected reward. Crucially, GRPO builds upon SFT in that, because SFT model already produces 'reasonable' outputs, when GRPO samples G outputs for the same query, some might be correct and some might not, creating a meaningful spread of rewards within the group. As described in **Question 1.**, those scoring above the group are reinforced; those below are suppressed.

SFT is preferred when high-quality reference data is available and correct outputs can be specified directly. GRPO is preferred when correctness is verifiable but many valid solution paths exist, such as mathematical reasoning, where imitation alone would never expose the model to the full space of correct behaviours.

Works Cited

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. (2017). Proximal policy optimization algorithms. *arXiv:1707.06347*.

Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Bi, X., Zhang, H., Zhang, M., Li, Y.K., Wu, Y., and Guo, D. (2024). DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv:2402.03300*.

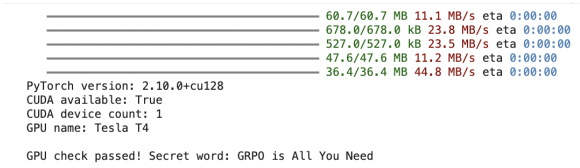
Skiredj, A. (2025). The Illustrated GRPO: A detailed and pedagogical explanation of the GRPO algorithm. Retrieved from https://abderrahmanskiredj.github.io/the-illustrated-grpo/.

Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., et al. (2022). Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35.

Part 2: Problems

Question 1.

The secret word is 'GRPO is All You Need.'



Question 3.

Problem 1: In your own words, explain why the group-based advantage normalization works as a baseline. What happens when all G completions for a prompt receive the same reward?

Group-vased advantage normalisation works as a baseline because it provides a natural reference point for what the model typically achieves on a given prompt. GRPO samples G outputs for the same query, scores each, and computes:

$$\hat{A}_i = \frac{r_i - \bar{r}}{\sigma_r + \epsilon},$$

where the group mean $\bar{r}$ *is* the baseline: if $r_i > \bar{r}$, output o_i was better than typical, so reinforce it; otherwise, suppress it. For example, suppose we sample four solutions to $\int_0^1 2x\,dx$ scoring $\{1, 1, 0, 0\}$. The correct solutions get $\hat{A}_{\{1,2\}} > 0$, whilst the incorrect get $\hat{A}_{\{3,4\}} < 0$, with no critic required.

Now, if all G outputs receive the same reward, then $\hat{A}_i = 0$ for all i. Thus, training signal vanishes and *no* update occurs. This highlights that GRPO requires a meaningful spread of rewards to exist within the group.

Problem 2: What is the role of the clipping term? What could go wrong during training without it?

Notice the probability ratio

$$\rho_{i,t} = \frac{\pi_\theta(o_{i,t}|q,\, o_{i,<t})}{\pi_{\theta_{\text{old}}}(oi, t|q,\, o_{i,<t})}$$

measures *how much* the current policy has changed relative to the old policy for a given token. The clipping term restricts this ratio to $[1-\epsilon, 1+\epsilon]$, for some sufficiently small $\epsilon > 0$, thereby inhibiting any single update from changing the policy too drastically. If we do not ensure updates remain conservative, an output with a large positive advantage $\hat{A}_i$ may cause π_θ to increase the probability of every token in that output (which share the same $\hat{A}_i$) by an unbounded amount in a one swift step. Such an overconfident update might lead to unstable training and collapse the diversity of outputs the model can generate.

Question 7.

1. Report the hyperparameter settings you used to get the best model. How did the reward values change over training?

The hyperparameters settings we used are recorded in Table 1. The best performing configuratio is Run 3, with NUM_GEN=2, TEMP=1.2, LR=1×10^{-5}, BETA=0.0, and LORA_R=16, achieving the lowest zero-signal rate of 70% and reward/mean of 1.300. Its initial and final rewards were 2.000 and 1.300 respectively, averaging 1.300 across all steps. This slight decline implies the model did not improve over training. Indeed, whilst higher temperature produces more diverse generations that created more learning signal (70% zero-signal), it also causes the model to drop the `Answer:` format (fmt/mean=0.890), pulling the combined reward down.

Run	NUM_GEN	TEMP	LR	BETA	LORA_R	% zero-signal	acc/mean	fmt/mean	reward/mean
Baseline	2	0.9	1e-5	0.0	16	84%	0.430	1.000	1.460
Run 2	**4**	0.9	1e-5	0.0	16	76%	0.400	1.000	1.400
Run 3	2	**1.2**	1e-5	0.0	16	70%	0.410	0.890	1.300
Run 4	2	0.9	**3e-5**	0.0	16	82%	0.490	0.980	1.470
Run 5	2	0.9	1e-5	**0.1**	16	86%	0.510	1.000	1.510
Run 6	2	0.9	1e-5	0.0	**8**	88%	0.470	0.990	1.460

Table 1: Hyperparameter experiments. Bold values indicate the single change from baseline per run.

Some observations:

- **Run 2 (NUM_GEN=4):** IN GRPO, learning signal arises *only* when generations within a group G differ in reward. With NUM_GEN=2, ties are likely, explaining the 84% of zero-signal steps in the baseline run. Increase to to 4 generations gives more chances for completions to differ in reward; this reduced zero-signal steps from 84% to 76%.
- **Run 3 (TEMP=1.2):** Recall temperature controls the randomness of the model's sampling, with higher values making generations more diverse and less likely to produce identical outputs. Observing 76% zero-signal steps remaining, generations are still too similar in reward. Raising temperature from 0.9 to 1.2 further reduced zero-signal steps to 70%; however, fmt/mean dropped to 0.890, suggesting higher temperature occasionally caused the model to drop the `Answer:` format.
- **Run 4 (LR=3e-5):** The learning rate controls how aggressively the model updates its weights on steps where signal exists. Since 16% of baseline steps did produce signal, a higher learning rate could exploit that signal more effectively. However, raising the learning rate from 1×10^{-5} to 3×10^{-5} had minimal effect on zero-signal steps (82% vs 84%), confirming that with so little signal to exploit, a higher learning rate does not help. acc/mean improved slightly from 0.430 to 0.490, but this is likely noise given the small dataset.
- **Run 5 (BETA=0.1):** The KL penalty penalises the model for deviating too far from the base model during training, acting as a regulariser to prevent reward hacking on a small dataset. With only 40 samples, unconstrained updates would risk overfitting to noise on the rare steps where signal exists. Therefore, we add BETA=0.1, which slightly increased zero-signal steps to 86% by reducing generation diversity. Note acc/mean marginally improved to 0.510, but this is likely noise; fmt/mean remained at 1.000, meaning the KL penalty preserved formatting behavior as expected.
- **Run 6 (LORA˙R=8):** LoRA rank controls the number of trainable parameters such that lower rank reduces model capacity and acts as a structural regulariser. With only 40 samples, we hypothesise rank 16 may be larger than necessary, possibly risking overfitting on the rare steps where signal exists. However, reducing rank from 16 to 8 slightly increased zero-signal steps to 88% and fmt/dropped to 0.990, suggesting that reduced capacity may have mildly impaired the model's ability to maintain consistent formatting.

2. Compare the training dynamics of GRPO (this homework) with SFT/LoRA (HW3). Which converged faster, and which produced better results on your dataset?
SFT/LoRA converged faster, and produced more stable and interpretable training dynamics than GRPO. Indeed, SFT training loss decreased steadily from the first epoch because every step produced a non-trivial gradient signal, enabling the model to directly minimise cross-entropy on correct outputs. Moreover, its loss curve diminished monotonically and its evaluation accuracy improved with more epochs and tuned LRs. As a result, the outputs were iteratively refined in format and confidence. Conversely, GRPO produced predominantly zero learning signal (see Table 1) across all runs. Since the base model `Qwen3-VL-2B-Instruct` already achieved near-perfect reward, generations consistently tied in reward, leaving reward functions unable to discriminate between completions and producing no meaningful gradient updates. As a result of the absence of reward constrast, GRPO failed to improve over the base model on this dataset.

Question 8.

Compare the GRPO-trained model with the pre-trained base model (zero-shot) and your SFT model from HW3: Does the GRPO model use the Answer: format more consistently? Does it show step-by-step reasoning before the answer?

The GRPO modes achieves 66.7$ accuracy on 3 test samples with 100% format compliance. It consistently produced step-by-step reasoning followed by a final `Answer:` label, enforced by format_reward during training.

The SFT/LoRA model produced concise single-word outputs but demonstrated clearer accuracy gains. In particular, it corrected out-of-vocabulary predictions from the base model (e.g. "Map" for deer, "Turtle" for frog) to correct `CIFAR-10` labels, showing SFT effectively aligned the model to the target label space. GRPO did not produce comparable accuracy gains, since the base model already performed well on most examples before training, leaving little room for improvement.

Overall, the two methods produced complementary effects: SFT improved label accuracy, while GRPO improved output format consistency.